Certifying global optima in non-convex optimization problems using deep neural networks

Charles Gulian *charlesgulian@berkeley.edu*
Industrial Engineering & Operations Research Dept.
University of California, Berkeley

Ying Chen *ying-chen@berkeley.edu*
Industrial Engineering & Operations Research Dept.
University of California, Berkeley

Javad Lavaei *lavaei@berkeley.edu*
Industrial Engineering & Operations Research Dept.
University of California, Berkeley

Reviewed on OpenReview: *https://openreview.net/forum?id=XXXX*

Abstract

Solving non-convex optimization problems in real-time is a ubiquitous challenge in cyber-physical systems engineering. Most fast solvers rely on heuristics or local search methods that are not guaranteed to converge to global optima for non-convex problems. On the other hand, slow, exact solvers such as convexification techniques or branch-and-bound are typically too slow to use in online settings. We propose a framework that bridges between fast and slow solvers via *neural network-based optimality certificates*. The framework leverages slow, exact solvers *offline* to learn the optimal value of parametric non-convex problems using deep neural networks. The neural network's predictions are then used *online* to certify global optimality of feasible solutions obtained from fast, local or heuristic solvers. We demonstrate the power of the proposed framework on several realistic problems from power systems, robotics, signal processing, and integer optimization. We also demonstrate its application to speed up traditional non-convex solvers such as branch-and-bound. Our results indicate that neural networks can effectively identify suboptimal solutions in real-time, potentially leading to improved solution accuracy without compromising speed.

1 Introduction

Quickly solving non-convex optimization problems is a ubiquitous challenge in many engineering disciplines. In power system operations, the AC optimal power flow (AC-OPF) problem is solved approximately every five minutes to balance time-varying load and generation (Van Hentenryck, 2021). Robots and autonomous vehicles solve non-convex route planning problems to reach targets and avoid collisions in changing environments (Ahmadi & Majumdar, 2016). In digital communication networks, signals are transmitted and recovered over noisy channels at millisecond timescales (Samuel et al., 2017). The list of applications goes on.

Many non-convex optimization solvers rely on fast but approximate algorithms or heuristics that may return suboptimal solutions, which incur higher cost, waste resources, or reduce safety margins unnecessarily. Therefore, being able to quickly certify the optimality of solutions obtained by fast solvers could potentially be extremely valuable. For example, operating the California power system costs roughly \$10-15 billion/year and emits roughly 40 MMT/year of greenhouse gases (CAISO, 2024). Even a 0.1% average improvement in operating efficiency could yield cost savings of \$10-15 million/year or reduce emissions by 40,000 MT/year.

However, certifying global optima is difficult for non-convex problems. Unlike in convex optimization, global optima in non-convex problems may not be certified from local information such as the KKT conditions (Boyd & Vandenberghe, 2004). In fact, certifying global optima in non-convex problems is NP-hard in general (Murty & Kabadi, 1985). A major challenge is the possible presence of multiple local optima where local search and gradient-based methods can get stuck. Stochastic search, ensemble methods, and various heuristic algorithms seek to overcome this issue by exploring the solution space more broadly and injecting noise to escape poor local optima (Dauphin et al., 2014; Kumar et al., 2020). Recently, machine learning-based solvers have also been used to obtain good solutions to non-convex problems (Donti et al., 2021; Jiang et al., 2024). Despite their impressive speed and accuracy, these methods are not guaranteed to converge to global optima, leading to worse outcomes in terms of cost, efficiency, or safety in real-world application settings.

Slow, exact solvers with global optimality guarantees for non-convex problems have also been studied extensively. These methods include spatial branch-and-bound solvers such as BARON and Couenne (Tawarmalani & Sahinidis, 2005; Belotti et al., 2009), as well as convexification methods such as sum-of-squares hierarchies and the reformulation-linearization technique (Lasserre, 2001; Ahmadi & Majumdar, 2019; Rudi et al., 2020; Sherali & Adams, 2013). Spatial branch-and-bound iteratively partitions the search space and solves convex relaxations over each subregion to obtain progressively tighter lower bounds. Convexification replaces non-convex objectives or constraints with convex outer-approximations, often in lifted or higher-dimensional spaces. Convex relaxations are solvable in polynomial time and provide global lower bounds for the optimal value of the original non-convex problem (Boyd & Vandenberghe, 2004). However, solving a convex relaxation can be exponentially slower than solving the original problem with local search or machine learning-based solvers, as convexification may introduce many more variables and constraints. Therefore, despite their power, these approaches scale poorly with problem size and are impractical to use in real-time applications.

We propose a framework which leverages machine learning to assist in non-convex optimization by certifying global optima in real-time. Specifically, we train deep neural networks to approximate the optimal value function of parametric non-convex problems. Convexification methods provide tight lower bounds on the optimal value of many problem instances, which are solved in parallel offline to obtain training data. The neural network predictions are then compared with the objective value of feasible solutions obtained from fast, local solvers to approximately certify global optimality. Our approach thus distills the power of convexification methods into *neural network optimality certificates* that can be deployed in real-time application settings.

1.1 Contributions

We make the following contributions in this work.

- We propose a novel machine learning framework that uses deep neural networks to certify global optima in non-convex optimization problems.
- We use convexification techniques to obtain tight lower bounds on the optimal value of non-convex problems, and thus obtain training data for our neural networks.
- We use split conformal calibration to obtain probabilistic guarantees for certification accuracy.
- We demonstrate the efficacy of our framework in several realistic experiments.

1.2 Paper Outline

In Section 2, we give an overview of related topics. In Section 3, we describe our framework and demonstrate it on a simple non-convex problem. In Section 4, we present results from several numerical experiments. In Section 5, we weigh the strengths and weaknesses of our framework. We conclude our work in Section 6.

1.3 Notation and Preliminaries

Throughout this paper, we denote scalars with unbolded lower-case letters z, vectors (and vector-valued functions) with bolded lower-case letters $\mathbf{z}$, and matrices with unbolded capital letters Z. We use $\|\cdot\|$ to denote the Euclidean norm and $|\cdot|$ to denote the absolute value norm.

2 Related Topics

Parametric Non-Convex Optimization

We are concerned with parametric non-convex optimization problems of the form

$$\begin{aligned} v(\boldsymbol{\theta}) \;=\; \min_{x \in \mathcal{X}} \quad & f(\mathbf{x}\,;\boldsymbol{\theta}) \\ \text{s.t.} \quad & \mathbf{x} \in \mathcal{C}(\boldsymbol{\theta}) \end{aligned} \tag{1}$$

where $\mathcal{C}(\boldsymbol{\theta}) = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{c}(\mathbf{x}\,;\boldsymbol{\theta}) = \mathbf{0}\}$ is a feasible set parameterized by $\boldsymbol{\theta} \in \Theta$. We assume that the objective function $f(\cdot\,;\boldsymbol{\theta})$ is non-convex or that the constraint functions $\mathbf{c}(\cdot\,;\boldsymbol{\theta})$ are nonlinear, making problem (1) a non-convex optimization problem. The parameter $\boldsymbol{\theta}$ typically represents some time-varying context or input to the optimization problem, such as electricity demand in optimal power flow or vehicle sensor data in route planning. Parametric problems are often encountered in cyber-physical engineering settings where a fixed non-convex problem must be solved repeatedly at high frequency for time-varying input data or parameters.

Non-convex problems are an extremely broad class of optimization problem. They are frequently encountered in real-world settings, and yet are notoriously difficult to solve. In fact, because general non-convex problems are NP-hard to solve, most successful solvers rely on bespoke algorithms tailored to specific problem classes; there are few generalist strategies. Indeed, designing efficient solvers for specific non-convex problems has remained an extremely active area of research in computer science and operations research for many years.

Despite their often impressive performance, fast non-convex solvers often lack guarantees of finding globally optimal solutions. Our goal in this work is to train deep neural networks to quickly *certify* global optima in non-convex problems. The neural networks are trained to predict the optimal value $v(\boldsymbol{\theta})$ of a parametric non-convex problem. Their predictions may therefore be compared with the objective value of any feasible solution $\hat{\boldsymbol{x}} \in \mathcal{C}(\boldsymbol{\theta})$ to estimate its optimality gap and thus certify whether $\hat{\boldsymbol{x}}$ is near-globally optimal. Our framework exploits convexification techniques and conformal prediction theory to provide probabilistic guarantees on the quality of a given solution and the accuracy of our certificates. The framework therefore *complements* the extensive body of work in non-convex optimization that has developed efficient solvers for a host of specific problems.

Machine Learning-Based Solvers

Machine learning-based solvers for non-convex problems have been proposed in a growing number of application areas, including power systems optimization, digital communications, and integer programming (Van Hentenryck, 2021; Samuel et al., 2017; Bertsimas & Stellato, 2022). Machine learning-based solvers often achieve very good performance on their respective target problems, and have been shown to be faster and more accurate than local search and heuristic algorithms in many cases. They appear to be promising alternatives to conventional solvers, yet they also suffer from the inability to guarantee or certify global optimality of obtained solutions. As a result, machine learning-based solvers have not been widely integrated into safety-critical applications (Sun et al., 2023). It remains an important challenge to develop similarly fast and accurate methods to *certify* optimal solutions obtained from machine learning-based solvers.

There are several approaches to designing machine learning-based solvers. One of the most common, which includes solvers such as *DeepOPF* (Pan et al., 2022), seeks to directly learn a mapping from parameters to optimal solutions of a non-convex problem. Machine learning-based solvers may be trained using supervised or self-supervised learning. In the former, the model is trained to directly minimize squared error loss between the predicted and true optimal solution. For models $\hat{\mathbf{x}}(\cdot) : \Theta \to \mathbb{R}^n$ in class $\mathcal{M}$, supervised learning solves

$$\min_{\hat{x}(\cdot) \in \mathcal{M}} \;\; \mathbb{E}_\theta \left\| \hat{\mathbf{x}}(\boldsymbol{\theta}) - \mathbf{x}^*(\boldsymbol{\theta}) \right\|^2 \tag{2}$$

where the expectation is taken with respect to some probability measure over Θ and $\mathbf{x}^*(\boldsymbol{\theta})$ is an optimal solution for problem (1). In self-supervised learning, the model is trained to directly minimize the objective, usually with some form of regularization to promote feasibility of predicted solutions.

$$\min_{\hat{x}(\cdot) \in \mathcal{M}} \;\; \mathbb{E}_\theta \left[\, f(\hat{\mathbf{x}}(\boldsymbol{\theta})\,;\boldsymbol{\theta}) + \rho \left\| \mathbf{c}(\hat{\mathbf{x}}(\boldsymbol{\theta})\,;\boldsymbol{\theta}) \right\|^2 \right] \tag{3}$$

The advantage of this approach is that it does not require labeled training data, which are expensive to obtain as it requires solving many non-convex optimization problems. A major challenge in either approach is that optimal solution mappings are high dimensional, nonlinear, and often discontinuous functions that are difficult to approximate well (Pan et al., 2022). Furthermore, it is difficult to balance optimality and feasibility conditions during training (Park & Van Hentenryck, 2023).

Other machine learning-based solvers integrate machine learning into traditional optimization algorithms. For example, Baker (2019) trains random forest models to predict warm start solutions to AC optimal power flow, which are then used to initialize local search solvers and accelerate their convergence. Other methods use machine learning to predict a partial subset of variables or active constraints in the optimal solution, which are then completed to obtain a full solution (Donti et al., 2021; Deka & Misra, 2019).

Unlike machine learning-based solvers which attempt to learn optimal solution mappings, our framework only requires learning the optimal value function $v(\boldsymbol{\theta})$. Even in non-convex problems, the optimal value function is typically much smoother than the optimal solution mapping, and furthermore, the prediction target remains a scalar regardless of the number of variables or constraints in the problem.

Convexification Hierarchies

A powerful approach to solving non-convex optimization problems is *convexification* (Zhang et al., 2018). The basic idea is to construct a convex relaxation of the original problem, in which the non-convex objective or constraints are replaced by convex outer-approximations. Convex relaxations often *lift* variables into higher dimensional spaces where the objective and constraints may be closely approximated by convex functions, and then *project* solutions in the lifted space back to the original space. Convex relaxations may be solved efficiently and with global optimality guarantees, but the dimension of the relaxed problem is often orders of magnitude larger, hence they are often slow to solve in practice for large problems. However, the accuracy of a convex relaxation often improves with the number of new variables and constraints introduced to approximate the original problem. This concept is directly embodied by *convexification hierarchies*, which are sequences of convex relaxations that can approximate non-convex problems to arbitrary accuracy.

Let $\mathcal{Y}_r$ be the (lifted) space of our convex relaxation, and let $\boldsymbol{L}_r : \mathcal{X} \to \mathcal{Y}_r$ be a map from the original space to the lifted space, and let $\boldsymbol{P}_r : \mathcal{Y}_r \to \mathcal{X}$ be a projection from the lifted space to the original space. Then the order-r convex relaxation of problem (1) in a convexification hierarchy (for $r = 0, 1, 2, \ldots$) is

$$\begin{aligned} v_r(\boldsymbol{\theta}) \;=\; \min_{y \in \mathcal{Y}_r} \quad & f_r(\mathbf{y}\,;\boldsymbol{\theta}) \\ \text{s.t.} \quad & \mathbf{y} \in \mathcal{C}_r(\boldsymbol{\theta}) \end{aligned} \tag{4}$$

where $f_r(\cdot\,;\boldsymbol{\theta})$ is a convex function and $\mathcal{C}_r(\boldsymbol{\theta}) \subseteq \mathcal{Y}_r$ is a convex set such that for any $\mathbf{x} \in \mathcal{C}(\boldsymbol{\theta})$ we have that $f_r(\boldsymbol{L}_r(\boldsymbol{x})\,;\boldsymbol{\theta}) \leq f(\mathbf{x}\,;\boldsymbol{\theta})$ and $\boldsymbol{L}_r(\mathbf{x}) \in \mathcal{C}_r(\boldsymbol{\theta})$. In other words, the relaxed objective under-approximates the original objective, and the relaxed feasible set contains the (lifted) original feasible set. It straightforward to see that the optimal value of a convex relaxation lower bounds the optimal value of the original problem: $v_r(\boldsymbol{\theta}) \leq v(\theta)$ for all $\boldsymbol{\theta} \in \Theta$. Furthermore, within a convexification hierarchy we have

$$v_0(\boldsymbol{\theta}) \leq v_1(\boldsymbol{\theta}) \leq v_2(\boldsymbol{\theta}) \leq \cdots \leq v(\boldsymbol{\theta})$$

$$\boldsymbol{P}_0(\mathcal{C}_0(\boldsymbol{\theta})) \supseteq \boldsymbol{P}_1(\mathcal{C}_1(\boldsymbol{\theta})) \supseteq \boldsymbol{P}_2(\mathcal{C}_2(\boldsymbol{\theta})) \cdots \supseteq \mathcal{C}(\boldsymbol{\theta})$$

with $v_r(\boldsymbol{\theta}) \to v(\boldsymbol{\theta})$ and $\mathcal{C}_r(\boldsymbol{\theta}) \to \mathcal{C}(\boldsymbol{\theta})$ as $r \to \infty$. In other words, the convex relaxations approximate the original problem to arbitrary accuracy as the order increases.

As a concrete example, consider the reformulation-linearization technique (RLT) (Sherali & Adams, 2013). RLT multiplies constraints together to generate new valid inequalities, and then linearizes terms by introducing auxiliary variables. For example, consider polynomial constraints of the form $g_i(\mathbf{x}) = \sum_{\boldsymbol{\alpha}} g_{i,\boldsymbol{\alpha}} \mathbf{x}^{\boldsymbol{\alpha}} \geq 0$ for $i \in \{1, \ldots l\}$ (and where $\boldsymbol{\alpha} \in \mathbb{N}^n$, $g_{i,\boldsymbol{\alpha}} \in \mathbb{R}$, and $\mathbf{x}^{\boldsymbol{\alpha}} := x_1^{\alpha_1} x_2^{\alpha_2} \cdots x_n^{\alpha_n}$). Multiplying any pair of polynomial constraints together yields a *valid inequality* $g_i(\mathbf{x}) g_j(\mathbf{x}) = \sum_{\boldsymbol{\alpha},\boldsymbol{\beta}} g_{i,\boldsymbol{\alpha}} g_{j,\boldsymbol{\beta}} \mathbf{x}^{\boldsymbol{\alpha}+\boldsymbol{\beta}} \geq 0$. Replacing monomial terms with auxiliary variables in a lifted space yields a linear relaxation with inequality constraints of the form $\sum_{\boldsymbol{\alpha},\boldsymbol{\beta}} g_{i,\boldsymbol{\alpha}} g_{j,\boldsymbol{\beta}} y_{\boldsymbol{\alpha}+\boldsymbol{\beta}} \geq 0$. Higher order relaxations within the hierarchy multiply larger sets of constraints, yielding progressively tighter relaxations with increasing numbers of variables and constraints.

Other convexification hierarchies use similar lifting and linearization procedures but introduce additional constraints on the lifted variables to yield tighter formulations. For example, the Lasserre (sum-of-squares) hierarchy introduces a lifted matrix variable Y (called the *moment matrix*) so that $Y_{\boldsymbol{\alpha},\boldsymbol{\beta}}$ replaces the monomial $\mathbf{x}^{\boldsymbol{\alpha}+\boldsymbol{\beta}}$, and additionally introduces a positive semidefinite cone constraint $Y \succeq 0$ motivated by the observation that $\mathbf{m}_d(\mathbf{x})\,\mathbf{m}_d(\mathbf{x})^\top \succeq 0$ for a degree-d monomial basis $\mathbf{m}_d(\mathbf{x}) := [\mathbf{x}^{\boldsymbol{\alpha}} : \mathbf{x} \in \mathbb{R}^n, \boldsymbol{\alpha} \in \mathbb{N}^n,\ |\boldsymbol{\alpha}| \leq d]$ (Lasserre, 2001). The resulting convex relaxation is a semidefinite program (SDP). More recently, Ahmadi & Majumdar (2016) introduced the closely related diagonally dominant sum-of-squares (DSOS/SDSOS) hierarchies, and Rudi et al. (2020) introduced the kernel sum-of-squares (kernel-SOS) hierarchy, which provide better computational tractability for high-order relaxations of large problems. Beugnot et al. (2023) developed a scalable, approximate non-convex solver based on the kernel-SOS hierarchy called *GloptiNets*, which produces its own probabilistic optimality certificates for obtained solutions.

Our framework uses convexification to obtain tight, provable lower bounds on the optimal value of parametric non-convex problems $v_r(\boldsymbol{\theta}) \leq v(\boldsymbol{\theta})$. The relaxed value function $v_r(\boldsymbol{\theta})$ is sampled by solving a convex relaxation for various $\boldsymbol{\theta} \in \Theta$ to obtain training data for a deep neural network. Convexification hierarchies allow us to trade off the tightness of the lower bound $v_r(\boldsymbol{\theta})$ and the computational cost of obtaining training data by increasing or decreasing the order of the convex relaxation.

Split Conformal Calibration

Split conformal calibration (Vovk et al., 2005; Lei et al., 2018) is a procedure for calibrating a machine learning model so that its predictions satisfy certain probabilistic performance guarantees. The procedure captures the intuitive notion that a model's performance on a heldout (unseen during training) test data set should be indicative of its expected performance on new test instances drawn from a similar distribution. Given a target $z(\boldsymbol{\theta})$ and trained predictive model $\hat{z}(\boldsymbol{\theta})$, let $(\boldsymbol{\theta}_1, z(\boldsymbol{\theta}_1)), \dots (\boldsymbol{\theta}_N, z(\boldsymbol{\theta}_N))$ be a heldout data set, and let $\boldsymbol{\theta}_{N+1}$ be a new test instance drawn from the same distribution. Conformal prediction constructs a *prediction set* $\mathcal{Z}_\alpha(\boldsymbol{\theta}_{N+1}) \subseteq \mathbb{R}$ that contains the true value of $z(\boldsymbol{\theta}_{N+1})$ with a user-specified probability

$$\mathbb{P}\big(\, z(\boldsymbol{\theta}_{N+1}) \in \mathcal{Z}_\alpha(\boldsymbol{\theta}_{N+1}) \,\big) \ \leq\ 1 - \alpha,$$

where $\alpha \in (0, 1)$ is a target miscoverage level and the probability is taken (marginalized) over the joint distribution of $(\boldsymbol{\theta}_1, z(\boldsymbol{\theta}_1)), \dots (\boldsymbol{\theta}_{N+1}, z(\boldsymbol{\theta}_{N+1}))$. For scalar targets, conformalized prediction normally constructs two-sided prediction intervals. However, for the purpose of constructing optimality certificates, our framework only requires one-sided prediction intervals. That is because we seek a *probabilistic lower bound* on the true optimal value $v(\boldsymbol{\theta})$ so that we can bound the optimality gap of candidate solutions. Concretely, we seek to construct a *conformal offset* $\hat{q}_\alpha \in \mathbb{R}$ from the heldout data such that

$$\mathbb{P}\big(\, z(\boldsymbol{\theta}_{N+1}) \geq \hat{z}(\boldsymbol{\theta}_{N+1}) - \hat{q}_\alpha \,\big) \ \geq\ 1 - \alpha.$$

The least conservative choice of $\hat{q}_\alpha$ which guarantees this coverage probability is the upper-α quantile of the prediction errors over the heldout data set

$$\hat{q}_\alpha \ :=\ \min\{\, q \in \mathbb{R} \,:\, \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}(\hat{z}(\boldsymbol{\theta}_i) - z(\boldsymbol{\theta}_i) \leq q) \ \geq\ 1 - \alpha \,\},$$

where $\mathbb{1}(\cdot)$ is the binary indicator function. Remarkably, this coverage guarantee holds with no assumptions about the model class, the input distribution, or the smoothness of the underlying function; for this reason conformal prediction is often called *distribution-free uncertainty quantification* (Angelopoulos & Bates, 2023). However, the coverage guarantee rests on one key assumption: the heldout data used to calibrate the model must be *exchangeable* with the new test instances. Informally, this means that they must be drawn from the same probability distribution. This assumption has important implications for the validity of our framework's probabilistic guarantees in practice, particularly under distribution shift, which we discuss in Section 5.

3 Our Framework

We propose a framework which uses deep neural networks to certify global optima in non-convex optimization problems. We train a deep neural network $\hat{v}(\boldsymbol{\theta})$ to approximate the relaxed optimal value function $v_r(\boldsymbol{\theta})$ (the

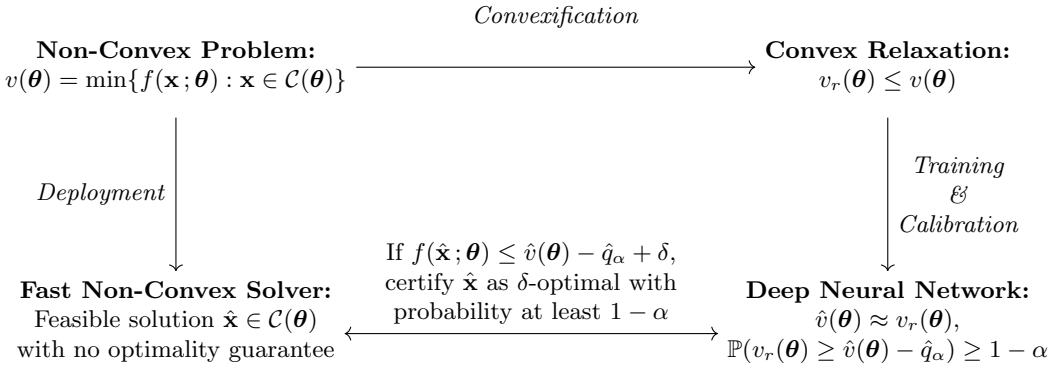

Figure 1: Our proposed approach for neural network-based optimality certification.

optimal value function of a convex relaxation) of a parametric non-convex problem. Given a new parameter instance $\boldsymbol{\theta} \in \Theta$, the neural network prediction $\hat{v}(\boldsymbol{\theta})$ may be compared with the objective value of any feasible solution $\hat{\mathbf{x}} \in \mathcal{C}(\boldsymbol{\theta})$ to estimate its optimality gap and thus certify whether $\hat{\mathbf{x}}$ is in fact near-globally optimal. We use convexification methods to trade off the tightness of convex relaxations with the computational cost of obtaining training data for the neural network. Furthermore, we use split conformal calibration to provide probabilistic guarantees on the accuracy of our optimality certificates. We envision that our framework should supplement fast, approximate solvers, including local search methods, heuristic algorithms, and machine learning-based solvers, by providing probabilistic optimality certificates in online settings.

Our framework is based on a well-known strategy for certifying global optima using convexification methods. For instance, Xu et al. (2021) propose to evaluate the KKT conditions of Lasserre hierarchy SDP relaxations at the candidate solution. Since the KKT conditions are sufficient to prove optimality in convex optimization, the candidate solution must be globally optimal if it satisfies the KKT conditions of the relaxed problem (Boyd & Vandenberghe, 2004). It also suffices to compare the optimal value of a convex relaxation with the objective value of the candidate solution. Solving a convex relaxation provides a lower bound on the otherwise intractable optimal value $v_r(\boldsymbol{\theta}) \leq v(\boldsymbol{\theta})$ for any $\boldsymbol{\theta} \in \Theta$. The objective value of a candidate solution $\hat{\mathbf{x}} \in \mathcal{C}(\boldsymbol{\theta})$ may thus be compared against $v_r(\boldsymbol{\theta})$ to upper bound its *optimality gap*, i.e. the gap between the solution's objective value and the optimal value:

$$\textit{optimality gap} \;:=\; f(\hat{\mathbf{x}};\boldsymbol{\theta}) - v(\boldsymbol{\theta}) \;\leq\; f(\hat{\mathbf{x}};\boldsymbol{\theta}) - v_r(\boldsymbol{\theta}).$$

Thus, if $f(\hat{\mathbf{x}};\boldsymbol{\theta}) - v_r(\boldsymbol{\theta}) \leq \delta$, then $f(\hat{\mathbf{x}};\boldsymbol{\theta}) - v(\boldsymbol{\theta}) \leq \delta$, certifying that $\hat{\mathbf{x}}$ is δ-optimal.

One obvious disadvantage of this strategy is that the tightness of the upper bound depends on the size of the *relaxation gap* $:= v(\boldsymbol{\theta}) - v_r(\boldsymbol{\theta})$, i.e. the gap between the original and relaxed problems' optimal values. It is non-trivial to bound the relaxation gap for an arbitrary convex relaxation of a non-convex problem. However, it is possible to control the relaxation gap by increasing the order of the convex relaxation within some convexification hierarchy (e.g., by introducing new valid inequalities or cutting planes). However, this typically would increase the size of the relaxed problem, making it slower to solve. This is a fundamental limitation of this strategy that prevents it from being useful in large-scale time-varying optimization settings. Our proposed framework overcomes this challenge by shifting the computational burden of solving large convex relaxations to an offline machine learning setting. The power of convexification methods may thus be distilled into deep neural networks whose predictions can certify global optima in real-time.

Our proposed framework has four main steps: (1) convexification, (2) training, (3) calibration, and (4) deployment. Convexification involves finding a suitable convex relaxation of a parametric non-convex problem. Training involves generating a training data set by solving many instances of the convex relaxation offline, and training a deep neural network to approximate the relaxed optimal value function. Calibration involves conformalizing the neural network's predictions to guarantee a user-specified coverage rate for the relaxed

optimal value. Finally, deployment involves choosing a certification threshold below which candidate solutions will be certified as near-globally optimal. We describe these steps in greater detail below.

1. Convexification Given a non-convex optimization problem (1) with value function $v(\boldsymbol{\theta})$, we construct a convex relaxation (4) with relaxed value function $v_r(\boldsymbol{\theta})$. By construction, $v_r(\boldsymbol{\theta}) \leq v(\boldsymbol{\theta})$ for all $\boldsymbol{\theta} \in \Theta$, i.e. solving the relaxation for any parameter instance yields a valid lower bound on the true optimal value, which is generally intractable. Within a convexification hierarchy, the order r of the relaxation governs a trade-off central to our framework: a higher-order relaxation is tighter (smaller relaxation gap $v(\boldsymbol{\theta}) - v_r(\boldsymbol{\theta})$) but slower to solve. Because the convexified problem is solved offline, we may choose the tightest relaxation that remains tractable at the scale of the training set, rather than the scale of real-time deployment.

2. Training We sample N parameter instances $\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_N$ from a probability distribution $\mathbb{P}(\cdot)$ over Θ and solve the convex relaxation once per instance to obtain training pairs $(\boldsymbol{\theta}_i, v_r(\boldsymbol{\theta}_i))$. A deep neural network $\hat{v}(\boldsymbol{\theta})$ is then trained to approximate the relaxed value function by minimizing empirical mean-squared error:

$$\min_{\hat{v}(\cdot)} \ \frac{1}{N} \sum_{i=1}^{N} \big(v_r(\boldsymbol{\theta}_i) - \hat{v}(\boldsymbol{\theta}_i) \big)^2$$

Unlike solvers that learn a mapping to an optimal solution $\mathbf{x}^*(\boldsymbol{\theta})$, the prediction target here is always a scalar, regardless of the dimension of the decision variable $\mathbf{x} \in \mathcal{X}$ or the number of constraints in $\mathcal{C}(\boldsymbol{\theta})$. This is one reason the value function tends to be an easier learning target in practice.

3. Calibration A trained network $\hat{v}(\boldsymbol{\theta})$ approximates $v_r(\boldsymbol{\theta})$ but offers no guarantee on any individual instance. We close this gap with split conformal calibration: using predictions on a held-out, exchangeable calibration set, we compute the one-sided conformal offset $\hat{q}_\alpha$ that satisfies

$$\mathbb{P}\big(\, v_r(\boldsymbol{\theta}) \geq \hat{v}(\boldsymbol{\theta}) - \hat{q}_\alpha \, \big) \geq 1 - \alpha$$

at a user-specified miscoverage level $\alpha \in (0, 1)$. The quantity $\hat{v}(\boldsymbol{\theta}) - \hat{q}_\alpha$ is then a calibrated, probabilistic lower bound on $v_r(\boldsymbol{\theta})$ that holds with probability at least $1 - \alpha$.

4. Deployment At runtime, we are given a parameter instance $\boldsymbol{\theta}$ and a feasible candidate solution $\hat{\mathbf{x}} \in \mathcal{C}(\boldsymbol{\theta})$ produced by any fast solver (local search method, heuristic algorithm, machine learning-based solver, etc.) with no optimality guarantee of its own. Evaluating $\hat{v}(\boldsymbol{\theta})$ costs a single forward pass through the neural network, which is orders of magnitude faster than re-solving the convex relaxation. We certify $\hat{\mathbf{x}}$ as δ-optimal, for a target optimality gap $\delta > 0$, whenever

$$f(\hat{\mathbf{x}}\,; \boldsymbol{\theta}) \ \leq \ \hat{v}(\boldsymbol{\theta}) - \hat{q}_\alpha + \delta.$$

Because $\hat{v}(\boldsymbol{\theta}) - \hat{q}_\alpha \leq v_r(\boldsymbol{\theta}) \leq v(\boldsymbol{\theta})$ holds with probability at least $1 - \alpha$, this certificate controls the probability of falsely declaring a candidate solution δ-optimal when its true optimality gap in fact exceeds δ. To see this, let us directly analyze the false positive rate (FPR) of the optimality certificate:

$$FPR \ := \ \mathbb{P}\big(\, f(\hat{\mathbf{x}}\,; \boldsymbol{\theta}) - \hat{v}(\boldsymbol{\theta}) \leq \delta - \hat{q}_\alpha \,|\, f(\hat{\mathbf{x}}\,; \boldsymbol{\theta}) - v(\boldsymbol{\theta}) > \delta \, \big).$$

By adding and subtracting $v(\boldsymbol{\theta})$, we may re-write this as

$$\mathbb{P}\big(\, f(\hat{\mathbf{x}}\,; \boldsymbol{\theta}) - v(\boldsymbol{\theta}) + v(\boldsymbol{\theta}) - \hat{v}(\boldsymbol{\theta}) \, \big) \leq \delta - \hat{q}_\alpha \,|\, f(\hat{\mathbf{x}}\,; \boldsymbol{\theta}) - v(\boldsymbol{\theta}) > \delta \, \big) \leq \mathbb{P}\big(\, \delta + v(\boldsymbol{\theta}) - \hat{v}(\boldsymbol{\theta}) \leq \delta - \hat{q}_\alpha \, \big),$$

which simplifies to $\mathbb{P}(\hat{v}(\boldsymbol{\theta}) - v(\boldsymbol{\theta}) \geq \hat{q}_\alpha)$. Since $v_r(\boldsymbol{\theta}) \leq v(\boldsymbol{\theta})$ for all $\theta \in \Theta$, we have

$$\mathbb{P}\big(\, \hat{v}(\boldsymbol{\theta}) - v(\boldsymbol{\theta}) \geq \hat{q}_\alpha \, \big) \leq \mathbb{P}\big(\, \hat{v}(\boldsymbol{\theta}) - v_r(\boldsymbol{\theta}) \geq \hat{q}_\alpha \, \big) \leq \alpha$$

by construction of $\hat{q}_\alpha$, hence the FPR of the certificate is bounded above by the miscoverage rate α.

We also note that there are potentially multiple combinations of miscoverage rate α and optimality gap δ that lead to the same certification threshold. We may certify $\hat{\mathbf{x}} \in \mathcal{C}(\boldsymbol{\theta})$ as δ-optimal with probability at least $1 - \alpha$ if $f(\hat{\mathbf{x}}\,; \boldsymbol{\theta}) \leq \hat{v}(\boldsymbol{\theta}) + \tau$ for any α, δ satisfying $\tau = \delta - \hat{q}_\alpha$. This lead to the notion of a *frontier* of probabilistic optimality certificates with the same cutoff but with varying levels of precision and accuracy.

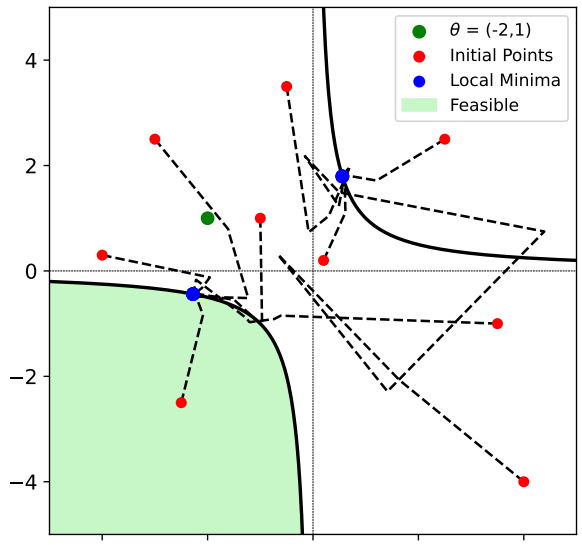

Figure 2: Interior point method convergence behavior for problem (5) for $\boldsymbol{\theta} = (-2, 1)$.

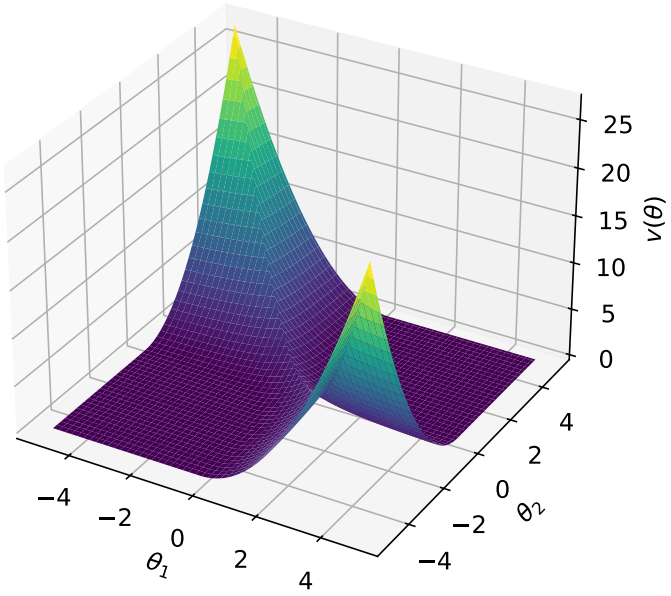

Figure 3: Optimal value function $v(\boldsymbol{\theta})$ over the parameter space $\Theta = [-5, 5] \times [-5, 5]$ for problem (5).

3.1 Example Problem: QCQP

Let us illustrate our framework in a simple problem setting. Consider the following parametric quadratically-constrained quadratic program (QCQP) over $x \in \mathbb{R}^2$:

$$\begin{aligned} v(\boldsymbol{\theta}) \;=\; \min_{x \in \mathbb{R}^2} \quad & (x_1 - \theta_1)^2 + (x_2 - \theta_2)^2 \\ \text{s.t.} \quad & x_1 x_2 \geq 1 \end{aligned} \tag{5}$$

In words, the problem is to find a point of minimum squared distance to $\boldsymbol{\theta} \in \mathbb{R}^2$ satisfying $x_1 x_2 \geq 1$. The quadratic constraint yields a non-convex feasible set with two disconnected regions. Therefore, any instance of this problem has two local minima.

As illustrated in Figure 2, the convergence behavior of local search methods depends strongly on the initial point used in the search. Some initial points converge to the global minimum while others converge to the spurious local minimum. A naive strategy would be to try multiple initial points and keep the best solution found. However, without a principled strategy for selecting initial points (or problem-specific knowledge about the number of local minima), it is not clear when to terminate this procedure. We now illustrate our framework's ability to certify global optima in this problem.

1. Convexification We shall convexify problem (5) via semidefinite relaxation. We lift the problem to the space of symmetric positive semidefinite (PSD) matrices. Entries of the lifted matrix variable $Y \in \mathbb{S}^{3\times 3}$ replace monomials in the objective and constraints of the original problem. For example, Y_{11} replaces $x_1^0 x_2^0 = 1$, Y_{22} replaces $x_1^2 x_2^0 = x_1^2$, and Y_{23} replaces $x_1^1 x_2^1 = x_1 x_2$. The semidefinite relaxation of problem (5) is therefore

$$\begin{aligned} v_r(\boldsymbol{\theta}) \;=\; \min_{Y \in \mathbb{S}^{3\times 3}} \quad & Y_{22} + Y_{33} - 2\theta_1 Y_{12} - 2\theta_2 Y_{13} + (\theta_1^2 + \theta_2^2) Y_{11} \\ \text{s.t.} \quad & Y_{11} = 1 \\ & Y_{23} \geq 1 \\ & Y \succeq 0 \end{aligned} \tag{6}$$

The relaxation (6) is always exact, i.e. $v_r(\boldsymbol{\theta}) = v(\boldsymbol{\theta})$ for all $\boldsymbol{\theta} \in \mathbb{R}^2$, because problem (5) has only one quadratic inequality constraint (see S-lemma; Boyd & Vandenberghe (2004)).

2. Training We sample 20,000 training instances and 5,000 test instances of $\boldsymbol{\theta}$ uniformly at random over $\Theta = [-5, 5] \times [-5, 5]$. We compute the relaxed optimal value $v_r(\boldsymbol{\theta})$ for each instance by solving problem (6). We then train deep neural networks $\hat{v}(\boldsymbol{\theta})$ to approximate $v_r(\boldsymbol{\theta})$ via four-fold cross-validation (see Section 4 for a detailed description of our neural network training procedure). The trained neural network achieves a mean absolute error (MAE) of 0.0022 over the test data set.

3. Calibration For each fold, we compute the upper α-quantile of the held-out set prediction errors, $\hat{q}_\alpha = 0.012$, for a chosen miscoverage rate of $\alpha = 1\%$, which we use to conformalize the neural network's predictions in the deployment phase to control the false positive rate.

4. Deployment We solve each problem instance in the test set with IPOPT, a local interior point solver, from a random fixed initialization (Wächter & Biegler, 2006). For each feasible local solution $\hat{\mathbf{x}}$ obtained by the local solver, we certify optimality if $f(\hat{\mathbf{x}}\,;\boldsymbol{\theta}) \leq \hat{v}(\boldsymbol{\theta}) + \delta - \hat{q}_\alpha$ for a chosen optimality tolerance $\delta = 0.138$. The optimality tolerance is calibrated to be $3\% \times \text{std.}\, v(\boldsymbol{\theta})$, so that we may certify optimality within 3% of the intrinsic variability of $v(\boldsymbol{\theta})$. The certification threshold is therefore $\delta - \hat{q}_\alpha = 0.126$. The neural network is extremely accurate relative to this certification threshold, and achieves zero false positive rate and near-zero false negative rate for certifying global optima over the test data set (results in Table 2 and Table 3).

4 Experiments

We evaluate our framework on four parametric non-convex problems drawn from robotics, power systems, digital communications, and integer programming.

Experimental Procedure

Each experiment instantiates the framework of Section 3. We fix a convex relaxation of the problem and generate training data by randomly sampling 20,000 training and 5,000 test parameters $\boldsymbol{\theta}_i \in \Theta$ and solving the relaxation for each to obtain $(\boldsymbol{\theta}_i, v_r(\boldsymbol{\theta}_i))$. We then train a four-fold ensemble of deep networks $\hat{v}(\boldsymbol{\theta})$ to approximate the relaxed value function $v_r(\boldsymbol{\theta})$ (Section 4) and calibrate each fold on its own held-out partition. Every test parameter is additionally solved by a fast local or heuristic solver to obtain the feasible solution $\hat{\mathbf{x}}(\boldsymbol{\theta})$ that the framework then certifies. We vary these ingredients across experiments, comparing convex relaxations of different orders (first- vs. second-order Lasserre SDP relaxation for inverse kinematics, SOCP vs. chordal SDP for AC-OPF), different fast solvers, different training sizes, and studying problem scales spanning three orders of magnitude.

We deploy the certificate of Section 3 (certifying $\hat{\mathbf{x}}$ as δ-optimal when $f(\hat{\mathbf{x}}\,;\boldsymbol{\theta}) \leq \hat{v}(\boldsymbol{\theta}) - \hat{q}_\alpha + \delta$) at a single fixed operating point throughout,

$$\delta \;=\; 3\% \times \text{std.}\, v(\boldsymbol{\theta}), \qquad \alpha \;=\; 0.01,$$

so that a certified solution is within 3% of the intrinsic variability of the optimal value $v(\boldsymbol{\theta})$ across instances, with 99% confidence (conformal offset $\hat{q}_{99}$). Scaling δ to the optimal value spread rather than a fixed relative optimality gap keeps the tolerance meaningful across problems of vastly different magnitude and avoids instability the instability of relative gaps when the optimal value is near zero. We score each certificate against the ground-truth event $f(\hat{\mathbf{x}}\,;\boldsymbol{\theta}) - v(\boldsymbol{\theta}) \leq \delta$, using the exact optimal value $v(\boldsymbol{\theta})$ where tractable and the relaxed optimal value $v_r(\boldsymbol{\theta})$ otherwise, and report true/false positive/negative counts (TP, FP, TN, FN) averaged over the four folds to demonstrate the accuracy of our framework in each problem setting.

Deep Neural Networks

Across all experiments we use a single network architecture and training recipe, selected by an ablation study on the AC-OPF problem (Appendix A). The network is a fully-connected feedforward network with six hidden layers of 256 ReLU units each, mapping the parameter $\boldsymbol{\theta}$ to a scalar value function estimate $\hat{v}(\boldsymbol{\theta})$. Network inputs and the regression target are standardized to zero mean and unit variance using statistics computed on the training partition, and predictions are mapped back to physical units at inference.

Figure 4: Training and validation loss (mean-squared error on the standardized target) for a representative fold model, on a logarithmic scale. The loss decays smoothly under the cosine-annealed schedule, with training and validation curves tracking closely.

We train for 1000 epochs with the AdamW optimizer (learning rate 10^{-3}, weight decay 10^{-4}, batch size 256) under a cosine-annealed learning-rate schedule. We partition the 20,000 training samples with 4-fold cross-validation (a fixed random shuffle, held constant across all experiments so that the split is reproducible), training one model per fold. Each fold's heldout validation partition serves as the conformal calibration set for that fold model, per step (4) above. A representative training history is shown in Figure 4: both the training and validation loss decay by roughly five orders of magnitude and track one another closely throughout training, indicating that the network fits the value function without overfitting.

4.1 Robotics: Inverse Kinematics

Inverse kinematics is a fundamental problem in robotics: given a target position for a robot's end effector, one must recover joint configurations that place the end effector at or as near as possible to the target. We consider a two-link planar arm with link lengths ℓ_1, ℓ_2 and joint angles ϕ_1, ϕ_2, parameterized by a target position $\boldsymbol{\theta} \in \mathbb{R}^2$. We reformulate this problem as a QCQP by lifting via trigonometric identities. Letting $c_i = \cos\phi_i$ and $s_i = \sin\phi_i$ for $i \in \{1, 2\}$, the end effector position is $(x_1, x_2) = \big(\ell_1 c_1 + \ell_2 c_{12},\ \ell_1 s_1 + \ell_2 s_{12}\big)$, where $c_{12} = c_1 c_2 - s_1 s_2$ and $s_{12} = s_1 c_2 + c_1 s_2$ by trigonometric identities. The problem of minimizing squared distance to the target is therefore the non-convex QCQP

$$
\begin{aligned}
v(\boldsymbol{\theta}) \;=\; \min_{(c,s)\in\mathbb{R}^4} \quad & (x_1-\theta_1)^2 + (x_2-\theta_2)^2 \\
\text{s.t.} \quad & c_1^2 + s_1^2 = 1 \\
& c_2^2 + s_2^2 = 1 \\
& x_1 = \ell_1 c_1 + \ell_2 (c_1 c_2 - s_1 s_2) \\
& x_2 = \ell_1 s_1 + \ell_2 (s_1 c_2 + c_1 s_2)
\end{aligned}
\tag{7}
$$

The quadratic equality constraints render the feasible set non-convex.

We convexify problem (7) by semidefinite lifting of the monomials in $\mathbf{y} = (c_1, s_1, c_2, s_2)$. The first-order Lasserre (Shor) relaxation introduces a moment matrix $Y \succeq 0$ with $Y_{00} = 1$ and linearizes the objective and the unit-circle constraints $c_i^2 + s_i^2 = 1$ in the entries of Y. The second-order Lasserre relaxation adds the degree-two moments for a tighter but larger SDP. We sample target positions $\boldsymbol{\theta}$ uniformly over a square covering the reachable annulus $[\,|\ell_1 - \ell_2|,\ \ell_1 + \ell_2\,]$ and slightly beyond, so that both reachable and unreachable

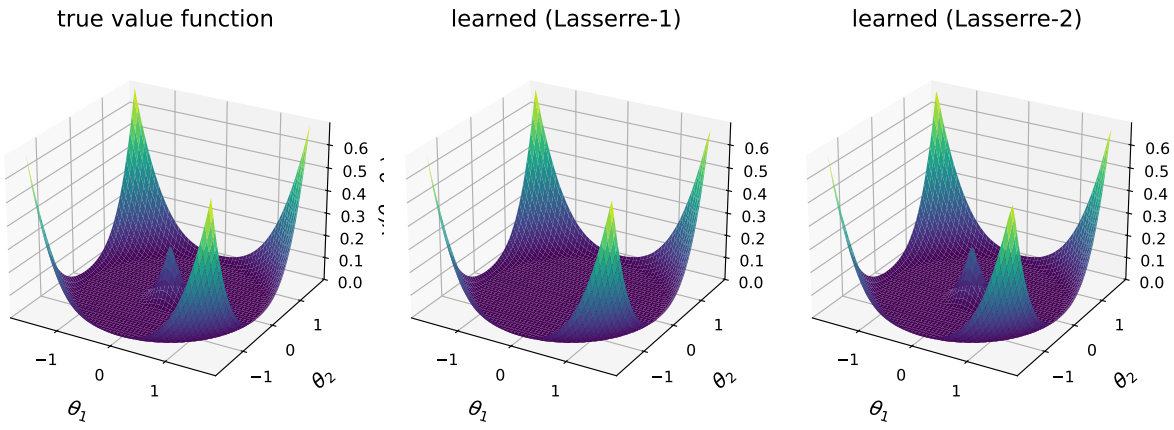

Figure 5: The true value function $v(\boldsymbol{\theta})$ for the inverse kinematics problem (left) and the neural network's learned approximation $\hat{v}(\boldsymbol{\theta})$ trained on the first-order Lasserre (Shor) relaxation (center) and the tighter second-order Lasserre relaxation (right), over target positions $\boldsymbol{\theta} = (\theta_1, \theta_2)$. All three panels share a common vertical scale and viewing angle. The flat zero-cost basin is the reachable workspace $|\ell_1 - \ell_2| \leq \|\boldsymbol{\theta}\| \leq \ell_1 + \ell_2$, where the target is attainable exactly, and the central spike is the unreachable region near the origin. Both learned surfaces closely reproduce the true value function, including the sharp central feature.

targets appear. We obtain feasible solutions by solving problem (7) with an interior point method (IPOPT). The ground-truth value function may be computed analytically in this experiment, which lets us exactly test the certification accuracy of our framework. Figure 5 visualizes this value function alongside the two learned approximations.

This experiment demonstrates an important aspect of our framework: the change from the first-order to second-order Lasserre hierarchy relaxation yields a tighter (exact) formulation that is more expensive to solve. This is a central tradeoff in our framework: tighter convex relaxations provide more accurate optimality certificates, but are more expensive to solve for generating training data. On the other hand, insufficient training data can worsen approximation error, leading to less accurate optimality certificates.

4.2 Digital Communications: MIMO Detection

The multiple-input multiple-output (MIMO) detection problem arises in digital communications, where a discrete signal must be recovered from a noisy linear measurement in real-time. A transmitted signal $\mathbf{x} \in \{-1, +1\}^n$ is sent over a fixed, complex-valued channel $H \in \mathbb{C}^{m \times n}$ and corrupted by Gaussian noise into the received signal $\boldsymbol{\theta} = H\mathbf{x} + \boldsymbol{\epsilon} \in \mathbb{C}^m$, which parameterizes the problem. Maximum-likelihood detection recovers $\mathbf{x}$ by minimizing the squared measurement error, yielding the non-convex QCQP

$$\begin{aligned} v(\boldsymbol{\theta}) \;=\; & \min_{x \in \mathbb{R}^n} \quad \|H\mathbf{x} - \boldsymbol{\theta}\|^2 \\ & \text{s.t.} \quad x_i^2 = 1, \quad i = 1, \ldots, n, \end{aligned} \tag{8}$$

whose 2^n feasible points (the candidate transmitted signals) are all local minima.

We convexify problem (8) via semidefinite relaxation (i.e., the Shor or Lasserre-1 relaxation). We introduce the lifted matrix variable Y such that Y is symmetric positive semidefinite and Y_{ij} replaces $x_i x_j$. We therefore require that $Y_{ii} = 1$ for all $i \in \{1, \ldots n\}$ in the constraints.

We fix a random channel matrix with $m = 4$, $n = 2$, sample transmitted signals $x \in \{-1, +1\}^n$ uniformly with noise variance $\sigma^2 = 0.1$, and obtain feasible solutions with a fast zero-forcing detector (a linear pseudo-inverse of H followed by rounding). MIMO detection is among the hardest of our value functions to learn: because the parameter is the noisy received signal, $v(\boldsymbol{\theta})$ jumps as $\boldsymbol{\theta}$ crosses the maximum-likelihood decision boundaries between candidate signals, so the target is rough rather than smooth. The network's prediction error is

consequently large relative to the small spread of the optimal value, which makes the certificate conservative for this problem and results in a high false negative rate. We return to this in Section 5.

4.3 Power Systems Optimization: AC Optimal Power Flow

The alternating current optimal power flow (AC-OPF) problem is solved by grid operators approximtely every five minutes to schedule generation at minimum cost subject to the physics of the electrical network. It is a non-convex QCQP parameterized by the real and reactive load $\boldsymbol{\theta} = (\mathbf{p}_D, \mathbf{q}_D)$ at each bus.

Let us now write the AC-OPF formulation. Consider a power network with a set of buses $\mathcal{N} = \{0, 1, 2, ..., n\}$, set of generator buses $\mathcal{G} \subseteq \mathcal{N}$, and set of lines $\mathcal{L} \subseteq \mathcal{N} \times \mathcal{N}$. We define the variables, parameters, and expressions associated with bus $k \in \mathcal{N}$ as follows:

- $v_k = v_{dk} + j\, v_{qk}$: Apparent (real and reactive) voltage.
- $p_{Dk} + j\, q_{Dk}$: Real and reactive power demand or load.
- $p_{Gk} + j\, q_{Gk}$: Real and reactive power generation.
- $f_{Ck}(p_{Gk}) = c_{k2}\, p_{Gk}^2 + c_{k1}\, p_{Gk} + c_{k0}$: Convex quadratic cost function of real power generation.
- $f_{vk}(\mathbf{v}_d, \mathbf{v}_q) = v_{dk}^2 + v_{qk}^2$: Squared voltage magnitude.

Let $Y = G + jB$ be the bus admittance matrix of an equivalent Π circuit model of the network. The real and reactive power generation at each bus are related to the bus voltages through the power flow equations $p_{Gk} = f_{Pk}(\mathbf{v}_d, \mathbf{v}_q)$ and $q_{Gk} = f_{Qk}(\mathbf{v}_d, \mathbf{v}_q)$, where

$$f_{Pk} := p_{Dk} + v_{dk} \sum_{i=1}^{n} (G_{ki} v_{di} - B_{ki} v_{qi}) + v_{qk} \sum_{i=1}^{n} (B_{ki} v_{di} + G_{ki} v_{qi}) \tag{9}$$

$$f_{Qk} := q_{Dk} + v_{dk} \sum_{i=1}^{n} (-B_{ki} v_{di} - G_{ki} v_{qi}) + v_{qk} \sum_{i=1}^{n} (G_{ki} v_{di} - B_{ki} v_{qi}) \tag{10}$$

We may write AC-OPF parameterized by real and reactive power demand $\boldsymbol{\theta} = (\mathbf{p}_D, \mathbf{q}_D)$ as follows:

$$\begin{aligned} v(\boldsymbol{\theta}) \;=\; \min \; & \sum_{k \in \mathcal{G}} f_{Ck}(f_{Pk}(\mathbf{v}_d, \mathbf{v}_q)) && (11) \\ \text{s.t. } & p_{Gk}^{min} \le f_{Pk}(\mathbf{v}_d, \mathbf{v}_q) \le p_{Gk}^{max} && (12) \\ & q_{Gk}^{min} \le f_{Qk}(\mathbf{v}_d, \mathbf{v}_q) \le q_{Gk}^{max} && (13) \\ & (v_k^{min})^2 \le f_{Vk}(\mathbf{v}_d, \mathbf{v}_q) \le (v_k^{max})^2 && (14) \\ & v_{q0} = 0 && (15) \end{aligned}$$

where constraints are defined for all buses $k \in \mathcal{N}$. Constraints (12) and (13) bound the real and reactive power output at each bus; constraint (14) bounds the voltage magnitude at each bus; and constraint (15) fixes the voltage angle of the slack bus to 0. In the full AC-OPF formulation used in our experiments, the magnitude of apparent power flow on lines is also constrained. We omit these constraints here for brevity.

For each test system we sample real and reactive load $\boldsymbol{\theta} = (\mathbf{p}_D, \mathbf{q}_D)$ and solve two convex relaxations of AC-OPF: the second-order cone (SOCP; Jabr, 2006) relaxation, and the tighter chordal semidefinite (SDP; Lavaei & Low, 2012) relaxation. Each load profile is generated by a two-level uniform perturbation of the network's nominal demand: a single system-wide scaling factor $\alpha \sim \text{Uniform}(0.6, 1.2)$ is drawn per sample, together with independent per-bus multipliers $\eta_i \sim \text{Uniform}(0.7, 1.3)$, so that $p_{Di} = \alpha\, \eta_i\, p_{Di}^0$ and $q_{Di} = \alpha\, \eta_i\, q_{Di}^0$, with the same η_i applied to real and reactive demand at bus i. The system-wide scaling factor captures a wide range of operating conditions, while the per-bus noise diversifies the demand pattern. Feasible solutions are obtained with an interior point method solver (IPOPT). The two convex relaxations again trade off computational cost for tightness: the SOCP is loose but fast to solve, whereas the chordal SDP tighter but far slower (Table 4). Because the true optimal value of AC-OPF is often intractable, we treat the chordal SDP relaxed optimal value as ground truth (for `case2869pegase`, whose SDP relaxation is also intractable at our time budget, we use the SOCP relaxed optimal value as ground truth). We study eight standard IEEE and PEGASE systems ranging from 9 buses to 2,869 buses (see Table 1).

Table 1: AC-OPF test systems used in our experiments.

System	Buses	Lines	Gens	Demand (MW)		
				nominal	min	max
`case9`	9	9	3	315	132	491
`case14`	14	20	5	259	109	404
`case39`	39	46	10	6254	2627	9757
`case89pegase`	89	206	12	8159	3427	12727
`case118`	118	179	54	4242	1782	6618
`case300`	300	409	69	23848	10016	37202
`case1354pegase`	1354	1710	260	74146	31141	115668
`case2869pegase`	2869	3968	510	138935	58353	216739

4.4 Integer Programming: Robust 0-1 Knapsack

The classical 0-1 knapsack problem selects items of value $\mathbf{c} \in \mathbb{R}^n$ and weight $\mathbf{w} \in \mathbb{R}^n$ to maximize their total value subject to a budget constraint $\mathbf{w}^\top \mathbf{x} \leq b$, where $\mathbf{x} \in \{0,1\}^n$. The *robust* 0-1 knapsack problem is a variant that considers *uncertain* weights $\mathbf{w} \in \mathcal{U}(\boldsymbol{\theta})$ belonging to a parametric uncertainty set. Here, we specifically consider *ellipsoidal* uncertainty sets with a fixed shape matrix $\boldsymbol{\Sigma} \succ 0$ and a varying center $\boldsymbol{\theta} = \boldsymbol{\mu} \in \mathbb{R}^n$,

$$\mathcal{U}(\boldsymbol{\theta}) \;=\; \{\boldsymbol{\mu} + \boldsymbol{\Sigma}^{1/2}\,\boldsymbol{\zeta} : \|\boldsymbol{\zeta}\| \leq \rho\},$$

where $\rho > 0$ is a fixed uncertainty budget. In robust 0-1 knapsack, the weight budget constraint must hold for every weight vector in the ellipsoidal uncertainty set:

$$\begin{array}{rcll} v(\boldsymbol{\theta}) & = & \max\limits_{x} & \mathbf{c}^\top \mathbf{x} \\ & & \text{s.t.} & \mathbf{w}^\top \mathbf{x} \leq b \quad \forall\, \mathbf{w} \in \mathcal{U}(\boldsymbol{\theta}) \\ & & & \mathbf{x} \in \{0,1\}^n \end{array} \tag{16}$$

Unlike other experiments, we do not seek a convex relaxation for problem (16). That is because this problem admits an exact reformulation as a mixed-integer second-order cone program (MI-SOCP):

$$\begin{array}{rcll} v(\boldsymbol{\theta}) & = & \max\limits_{x} & \mathbf{c}^\top \mathbf{x} \\ & & \text{s.t.} & \boldsymbol{\mu}^\top \mathbf{x} + \rho\,\|\boldsymbol{\Sigma}^{1/2}\mathbf{x}\| \leq b, \quad \mathbf{x} \in \{0,1\}^n \end{array} \tag{17}$$

This type of convex mixed integer nonlinear program that can be solved via standard techniques such as branch-and-bound. However, in real-time settings (such as in high-frequency trading), it may be necessary to use fast heuristics such as the greedy algorithm to obtain approximate solutions to problem (17).

In our experiment, we consider robust 0-1 knapsack with $n = 25$ items. The item values $\mathbf{c}$ (fixed, drawn once from Uniform$(20, 30)$), the budget $b = \frac{1}{2}\sum_i c_i$, the robustness parameter $\rho = 1$, and the shape matrix

$$\boldsymbol{\Sigma} \;=\; \tau^2\big[(1-\gamma)\mathbf{I} + \gamma\,\mathbf{1}\mathbf{1}^\top\big], \qquad \tau = 5, \;\; \gamma = 0.8,$$

are held constant across all instances, so that the only quantity varying across instances is the ellipsoid center: $\boldsymbol{\theta} = \boldsymbol{\mu} \in \mathbb{R}^{25}$. Each instance draws $\boldsymbol{\mu} \sim \mathcal{N}(\mathbf{c}, \boldsymbol{\Sigma})$, centering the nominal weights on the values so that more valuable items are also heavier on average and the selection problem is genuinely combinatorial.

The correlation γ deserves comment, because it is what makes this experiment informative. If the entries of $\boldsymbol{\theta}$ are drawn independently, the optimal value concentrates sharply: $v(\boldsymbol{\theta})$ is a sum over the $\sim n/2$ selected items, whose independent fluctuations average out, leaving a coefficient of variation of only a few percent and a value function that is essentially a constant plus combinatorial noise. A positive pairwise correlation defeats this averaging by introducing a common factor—all weights are heavy together, tightening the budget, or

light together, loosening it—so the total weight that fits, and hence $v(\boldsymbol{\theta})$, varies substantially across instances. At $\gamma = 0.8$ the coefficient of variation of v is 0.167, roughly four times its value in the independent case, while the greedy heuristic still finds the global optimum on only 7.4% of instances, leaving a large and genuinely suboptimal negative class to discriminate.

For each instance we solve problem (17) to certified global optimality with a branch-and-bound solver from Gurobi (Gurobi Optimization, LLC, 2024), and we additionally solve its continuous relaxation ($\mathbf{x} \in [0,1]^n$), a second-order cone program, with CLARABEL (Goulart & Chen, 2024). This yields two candidate learning targets: the exact optimal value $v(\boldsymbol{\theta})$, which is discontinuous in $\boldsymbol{\theta}$, and the relaxation value $v_r(\boldsymbol{\theta}) \geq v(\boldsymbol{\theta})$, which is continuous and therefore easier to fit, at the price of a relaxation gap that adds directly to the certificate's slack. We report both. The fast solver we certify is a greedy heuristic that ranks items by their value-to-nominal-weight ratio c_i/μ_i in descending order and adds them one at a time, keeping each addition only so long as the robust capacity constraint $\boldsymbol{\mu}^\top \mathbf{x} + \rho \left\| \boldsymbol{\Sigma}^{1/2} \mathbf{x} \right\| \leq b$ remains satisfied. Because the value function of a combinatorial problem is comparatively hard to learn, we repeat the experiment with 20,000 and 80,000 training instances, the smaller set being a subset of the larger.

4.5 Results

Tables 2–4 report all results at the fixed operating point $\delta = 3\% \times \text{std.}\, v(\boldsymbol{\theta})$ and $\alpha = 0.01$. Table 2 covers the offline stages: the mean and spread of the (relaxed) optimal value, the relaxation gap, the mean absolute error of the trained neural network predictions, and the calibrated conform offset $\hat{q}_{99}$. Table 3 reports the deployment results: the local solver's optimality gap and the certification counts. Across every experiment the false positive rate is near-zero or close to the 1% target, while the number of certified optima (TP) is high wherever the network is accurate enough that $\hat{q}_{99}$ fits within δ (AC-OPF, inverse kinematics, QCQP), and low where it is not (MIMO detection, and to a lesser extent robust knapsack), whose rougher value functions the network fits less precisely. Table 4 shows the offline solve cost: higher-order relaxations are consistently, and often dramatically, slower, and increasingly so as the system grows—on AC-OPF the chordal SDP costs under 3× the SOCP on `case9` but is 19–107× slower on `case118`–`case89pegase`, remaining over 40× slower even on the much larger `case1354pegase`—whereas a network prediction is sub-microsecond. The local solver, by contrast, does not show a comparably clean scaling trend relative to the SOCP relaxation (Table 4); we caution against over-reading this comparison given the sample size.

5 Discussion

Our results indicate that our framework achieves its intended purpose of probabilistically certifying global optima in non-convex optimization problems. In experiments where the deep neural network learns to approximate the value function with high accuracy, it can certify global optima at or below the target false positive rate of 1% within only a few percent of the intrinsic spread of the target optimal value. For problems with more challenging value functions, the certification accuracy improves with larger training data – illustrating the "preservation of difficulty" of certain NP-hard problems. As a feedforward pass through the neural network takes only milliseconds, the certificates may be deployed to assist fast solvers (including local search methods, heuristics, or machine learning-based solvers) in real-time settings.

The probabilistic guarantees of our framework rest on *exchangeability* between the calibration sample and new test instances. This holds when parameters are drawn i.i.d. from a fixed distribution, as in our experiments. In realistic, time-varying optimization settings, where observed instances of $\boldsymbol{\theta}$ may be temporally correlated or subject to distribution shift, the marginal coverage of our framework can degrade, and the calibrated conformal offset should be refreshed online or replaced by a weighted conformal variant (where calibration samples are re-weighted to account for covariate shift) (Tibshirani et al., 2019).

Furthermore, coverage probability in split conformal calibration is *marginal*: it holds on average over Θ, not conditionally at a particular $\boldsymbol{\theta}$, so the error rate may be uneven across the parameter space. Conformalized quantile regression (Romano et al., 2019), which calibrates a learned quantile model rather than a single conformal offset, is a natural route to approximate conditional coverage and an interesting direction for future work.

Table 2: Offline results (data generation, convexification, training, and split-conformal calibration), averaged over the four folds. *If $v(\boldsymbol{\theta})$ is intractable we use the relaxation value $v_r(\boldsymbol{\theta})$ for the mean, std, and relaxation gap.

			Data Generation		Convexification	Training	Calibration
Experiment	**Relaxation**	**Case**	Mean $v_r(\boldsymbol{\theta})$	Std $v_r(\boldsymbol{\theta})$	Relax. Gap*	MAE	q_{99}
QCQP	Shor		2.65	4.63	0	0.0022	0.012
Inverse Kinematics	Shor		0.036	0.094	0.0029	7.7e-05	0.00055
	Lasserre-2		0.039	0.094	0	8.4e-05	0.00053
MIMO Detection	Shor		0.79	0.41	0.00045	0.017	0.057
AC-OPF	SOCP	`case9`	4,692	1,452	0.04	0.72	2.79
	SOCP	`case14`	7,049	1,906	5.8	2.82	17.5
	SOCP	`case39`	33,515	12,532	6.6	33.2	219
	SOCP	`case89pegase`	7,274	1,410	6.46	2.52	19.4
	SOCP	`case118`	113,180	28,637	272	27.8	270
	SOCP	`case300`	586,141	133,013	803	267	2,325
	SOCP	`case1354pegase`	67,385	13,340	43.7	4.17	18.4
	SOCP	`case2869pegase`	126,542	24,379	0	7.98	33.2
	SDP	`case9`	4,692	1,452	0	0.73	2.71
	SDP	`case14`	7,054	1,907	0	2.94	18.2
	SDP	`case39`	33,522	12,533	0	30.8	196
	SDP	`case89pegase`	7,281	1,412	0	2.52	19.1
	SDP	`case118`	113,452	28,705	0	28.4	281
	SDP	`case300`	583,523	130,830	0	237	1,485
	SDP	`case1354pegase`	67,676	13,279	0	4.84	19.4
Robust Knapsack	Exact (Integer)	`n=25`	281	46.9	0	1.23	3.96
	SOCP	`n=25`	281	46.9	1.71	1.16	3.66

Finally, we also note that the *power* of our certificates is inherited from the tightness of the convex relaxation used to generate training data. Intuitively, a large relaxation gap results in an overly conservative certification threshold. This is because the candidate solution's objective value $f(\hat{\mathbf{x}}\,;\boldsymbol{\theta})$ is bounded below by $v(\boldsymbol{\theta})$, but the certification threshold is a calibrated probabilistic lower bound for $v_r(\boldsymbol{\theta})$. Since the relaxation gap may also be uneven across the parameter space (as observed in experiment 4.1), hence the conservativeness of the certificates (and the false negative rate) may be uneven as well. Tightening the relaxation (e.g. by moving up a convexification hierarchy or adding valid inequalities or cutting planes to the formulation) directly improves certification accuracy at a stricter optimality tolerance, at the offline cost of a larger convex relaxation.

6 Conclusion

In this work, we proposed an innovative framework for real-time optimality certification in non-convex optimization tasks. Our framework leverages convexification techniques and machine learning to learn the optimal value function of parametric optimization problems. By functioning as a meta-certification framework, our method is compatible with various convexification schemes as well as various fast, approximate solvers. By training neural networks to approximate the optimal value of a convex relaxation, we shift the computational burden from real-time optimization to offline training. Under suitable conditions, the convex relaxation provides a tight lower bound on the original non-convex problem, so candidate solutions that are sufficiently close to the predicted optimal value are assured to be near-globally optimal. By conformalizing our network predictions, we are able to guarantee coverage probability and bound the misclassification rate (under certain assumptions like exchangeability of the data) Our proposed framework not only expands the applicability of convexification methods to online optimization, but also offers a new conceptual approach to apply learning-based performance guarantees to fast heuristic solvers. Future work will focus on refining machine learning-based optimality certification using conformal quantile regression, and extending this approach to more diverse classes of non-convex problems.

Table 3: Online (deployment) results at $\delta = 3\% \cdot \mathrm{std}(v(\boldsymbol{\theta}))$ and $\alpha = 1\%$ (offset q_{99}), averaged over the four folds. *Optimality gap uses $v(\boldsymbol{\theta})$ where tractable, else $v_r(\boldsymbol{\theta})$.

			Deployment					
Experiment	**Relaxation**	**Case**	Mean / Worst Opt. Gap*	δ	TP	FP	TN	FN
QCQP	Shor		9.76 / 71.1	0.139	2524	0	2474	2
Inverse Kinematics	Shor		0.011 / 2	0.00282	4533	0	193	267
	Lasserre-2		0.011 / 2	0.00282	4799	0	193	1
MIMO Detection	Shor		0.0019 / 2.64	0.0122	96	0	4	4900
AC-OPF	SOCP	`case9`	4e-05 / 0.0014	43.6	5000	0	0	0
	SOCP	`case14`	0.0057 / 5.15	57.2	4999	0	0	1
	SOCP	`case39`	4.11 / 351	376	4542	0	0	109
	SOCP	`case89pegase`	0.026 / 0.33	42.4	4698	0	0	75
	SOCP	`case118`	2.01 / 4.82	861	4952	0	0	48
	SOCP	`case300`	235 / 9,692	3,925	3702	0	9	232
	SOCP	`case1354pegase`	5.96 / 32.9	398	3844	0	0	0
	SOCP	`case2869pegase`	260 / 6,538	731	2957	0	259	0
	SDP	`case9`	4e-05 / 0.0014	43.6	5000	0	0	0
	SDP	`case14`	0.0057 / 5.15	57.2	5000	0	0	0
	SDP	`case39`	4.11 / 351	376	4584	0	0	67
	SDP	`case89pegase`	0.026 / 0.33	42.4	4750	0	0	28
	SDP	`case118`	2.01 / 4.82	861	4995	0	0	5
	SDP	`case300`	235 / 9,692	3,925	3894	0	9	39
	SDP	`case1354pegase`	5.96 / 32.9	398	3855	0	0	0
Robust Knapsack	Exact (Integer)	`n=25`	9.02 / 24.5	1.41	50	4	4238	708
	SOCP	`n=25`	9.02 / 24.5	1.41	7	1	4241	751

Broader Impact Statement

In this optional section, TMLR encourages authors to discuss possible repercussions of their work, notably any potential negative impact that a user of this research should be aware of. Authors should consult the TMLR Ethics Guidelines available on the TMLR website for guidance on how to approach this subject.

Author Contributions

If you'd like to, you may include a section for author contributions as is done in many journals. This is optional and at the discretion of the authors. Only add this information once your submission is accepted and deanonymized.

Acknowledgments

Use unnumbered third level headings for the acknowledgments. All acknowledgments, including those to funding agencies, go at the end of the paper. Only add this information once your submission is accepted and deanonymized.

References

Amir Ali Ahmadi and Anirudha Majumdar. Some applications of polynomial optimization in operations research and real-time decision making. *Optimization Letters*, 10:709–729, 2016.

Amir Ali Ahmadi and Anirudha Majumdar. Dsos and sdsos optimization: more tractable alternatives to sum of squares and semidefinite optimization. *SIAM Journal on Applied Algebra and Geometry*, 3(2):193–230, 2019.

Anastasios N Angelopoulos and Stephen Bates. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 16(4):494–591, 2023.

Table 4: Mean per-instance solve times: the convex relaxation (offline) vs. the fast local solver. Both columns of a given row are measured identically, so each row is a like-for-like comparison; the local solver does not depend on which relaxation it is being compared against, so its solve time is measured ONCE per experiment (per AC-OPF case; once for inverse kinematics) and reused across every relaxation row of that experiment, rather than re-measured independently per row. For QCQP, inverse kinematics and MIMO detection we report the full wall-clock of each solve call. For AC-OPF, whose local solver (IPOPT) is invoked through a file-based (AMPL/NL) modelling interface, we report *solver-internal* time on both sides—the conic solver's own reported solve time for the relaxation, and IPOPT's own reported solve time for the local solve (parsed from its internal timing log)—so that the comparison is not dominated by Python model construction, a power-flow warm start, or the NL-file read/write and process-launch overhead of the modelling interface, which together account for a further 2–7× on the local side for the smallest systems (diluted to ∼1.3× on the largest). Mean over 50 random test instances per case (smallest six AC-OPF cases) or 3–8 instances for `case1354pegase`/`case2869pegase`'s SOCP and `case1354pegase`'s chordal SDP, whose solve is tens of seconds per instance; instances on which either solver failed to converge are excluded from both columns (up to 26% of draws for the chordal SDP on the largest cases, itself indicative of that relaxation's numerical difficulty there). †For `case2869pegase`'s chordal SDP, only 1 of 5 attempted instances converged (each attempt costing ∼70s regardless of outcome); we report that single solve time rather than assert a stable average, consistent with treating this relaxation as effectively intractable at scale (§4.3). Within a relaxation family, cost grows with problem size, and the chordal SDP is far more expensive than the SOCP, especially at scale; the SOCP-vs.-local gap does not show a comparably clean scaling trend and should not be over-read from this sample size.

Experiment	**Relaxation**	**Case**	Relaxation Solve	Local Solve
QCQP	Shor		96.1 ms	94.7 ms
Inverse Kinematics	Shor		0.36 ms	12.4 ms
	Lasserre-2		34.5 ms	12.4 ms
MIMO Detection	Shor		92.8 ms	0.141 ms
AC-OPF	SOCP	`case9`	1.80 ms	4.40 ms
	SOCP	`case14`	0.60 ms	3.60 ms
	SOCP	`case39`	2.40 ms	20.6 ms
	SOCP	`case89pegase`	10.4 ms	18.7 ms
	SOCP	`case118`	6.80 ms	19.8 ms
	SOCP	`case300`	20.0 ms	41.0 ms
	SOCP	`case1354pegase`	128 ms	247 ms
	SOCP	`case2869pegase`	349 ms	467 ms
	SDP	`case9`	2.10 ms	4.40 ms
	SDP	`case14`	5.00 ms	3.60 ms
	SDP	`case39`	27.4 ms	20.6 ms
	SDP	`case89pegase`	1.10 s	18.7 ms
	SDP	`case118`	128 ms	19.8 ms
	SDP	`case300`	787 ms	41.0 ms
	SDP	`case1354pegase`	8.28 s	247 ms
	SDP	`case2869pegase`	40.3 s†	467 ms
Robust Knapsack	Exact (Integer)	`n=25`	–	0.0687 ms
	SOCP	`n=25`	–	0.0687 ms

Kyri Baker. Learning warm-start points for ac optimal power flow. In *2019 IEEE 29th International Workshop on Machine Learning for Signal Processing (MLSP)*, pp. 1–6. IEEE, 2019.

Pietro Belotti, Jon Lee, Leo Liberti, François Margot, and Andreas Wächter. Branching and bounds tighteningtechniques for non-convex minlp. *Optimization Methods & Software*, 24(4-5):597–634, 2009.

Dimitris Bertsimas and Bartolomeo Stellato. Online mixed-integer optimization in milliseconds. *INFORMS Journal on Computing*, 34(4):2229–2248, 2022.

Gaspard Beugnot, Julien Mairal, and Alessandro Rudi. Gloptinets: Scalable non-convex optimization with certificates, 2023. URL `https://arxiv.org/abs/2306.14932`.

Stephen Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge university press, 2004.

CAISO. 2023 annual report on market issues and performance. Annual report, California Independent System Operator, August 2024. URL `https://www.caiso.com/library/market-issues-and-performance-annual-reports`.

Yann N Dauphin, Razvan Pascanu, Caglar Gulcehre, Kyunghyun Cho, Surya Ganguli, and Yoshua Bengio. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. *Advances in neural information processing systems*, 27, 2014.

Deepjyoti Deka and Sidhant Misra. Learning for dc-opf: Classifying active sets using neural nets. In *2019 IEEE Milan PowerTech*, pp. 1–6. IEEE, 2019.

Priya L Donti, David Rolnick, and J Zico Kolter. Dc3: A learning method for optimization with hard constraints. *arXiv preprint arXiv:2104.12225*, 2021.

Paul J. Goulart and Yuwen Chen. Clarabel: An interior-point solver for conic optimization problems. `https://github.com/oxfordcontrol/Clarabel.rs`, 2024. Software.

Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2024. URL `https://www.gurobi.com`.

R. A. Jabr. Radial distribution load flow using conic programming. *IEEE Transactions on Power Systems*, 21(3):1458–1459, 2006.

Bozhen Jiang, Qin Wang, Shengyu Wu, Yidi Wang, and Gang Lu. Advancements and future directions in the application of machine learning to ac optimal power flow: a critical review. *Energies*, 17(6):1381, 2024.

Abhishek Kumar, Guohua Wu, Mostafa Z Ali, Rammohan Mallipeddi, Ponnuthurai Nagaratnam Suganthan, and Swagatam Das. A test-suite of non-convex constrained optimization problems from the real-world and some baseline results. *Swarm and Evolutionary Computation*, 56:100693, 2020.

Jean B Lasserre. Global optimization with polynomials and the problem of moments. *SIAM Journal on optimization*, 11(3):796–817, 2001.

Javad Lavaei and Steven H Low. Zero duality gap in optimal power flow problem. *IEEE Transactions on Power Systems*, 1(27):92–107, 2012.

Jing Lei, Max G'Sell, Alessandro Rinaldo, Ryan J Tibshirani, and Larry Wasserman. Distribution-free predictive inference for regression. *Journal of the American Statistical Association*, 113(523):1094–1111, 2018.

Katta G Murty and Santosh N Kabadi. Some np-complete problems in quadratic and nonlinear programming. Technical report, 1985.

Xiang Pan, Minghua Chen, Tianyu Zhao, and Steven H Low. Deepopf: A feasibility-optimized deep neural network approach for ac optimal power flow problems. *IEEE Systems Journal*, 17(1):673–683, 2022.

Seonho Park and Pascal Van Hentenryck. Self-supervised primal-dual learning for constrained optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 4052–4060, 2023.

Yaniv Romano, Evan Patterson, and Emmanuel Candès. Conformalized quantile regression. *Advances in Neural Information Processing Systems*, 32, 2019.

Alessandro Rudi, Ulysse Marteau-Ferey, and Francis Bach. Finding global minima via kernel approximations. *arXiv preprint arXiv:2012.11978*, 2020.

Neev Samuel, Tzvi Diskin, and Ami Wiesel. Deep mimo detection. In *2017 IEEE 18th International Workshop on Signal Processing Advances in Wireless Communications (SPAWC)*, pp. 1–5. IEEE, 2017.

Hanif D Sherali and Warren P Adams. *A reformulation-linearization technique for solving discrete and continuous nonconvex problems*, volume 31. Springer Science & Business Media, 2013.

Bo Sun, Jerry Huang, Nicolas Christianson, Mohammad Hajiesmaili, Adam Wierman, and Raouf Boutaba. Online algorithms with uncertainty-quantified predictions. *arXiv preprint arXiv:2310.11558*, 2023.

Mohit Tawarmalani and Nikolaos V Sahinidis. A polyhedral branch-and-cut approach to global optimization. *Mathematical programming*, 103(2):225–249, 2005.

Ryan J. Tibshirani, Rina Foygel Barber, Emmanuel Candès, and Aaditya Ramdas. Conformal prediction under covariate shift. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Pascal Van Hentenryck. Machine learning for optimal power flows. *Tutorials in Operations Research: Emerging Optimization Methods and Modeling Techniques with Applications*, pp. 62–82, 2021.

Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer, 2005.

Andreas Wächter and Lorenz T Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical programming*, 106(1):25–57, 2006.

Sikun Xu, Ruoyi Ma, Daniel K Molzahn, Hassan Hijazi, and Cédric Josz. Verifying global optimality of candidate solutions to polynomial optimization problems using a determinant relaxation hierarchy. In *2021 60th IEEE Conference on Decision and Control (CDC)*, pp. 3143–3148. IEEE, 2021.

Richard Y Zhang, Cédric Josz, and Somayeh Sojoudi. Conic optimization theory: Convexification techniques and numerical algorithms. In *2018 Annual American Control Conference (ACC)*, pp. 798–815. IEEE, 2018.

A Neural Network Ablation Study

The architecture (6 hidden layers of 256 units) and training budget (1000 epochs) used throughout the paper were selected by ablation on the AC-OPF problem, varying one factor at a time and measuring 4-fold cross-validated prediction accuracy on the SOCP relaxation. We report the mean absolute percent error (MAPE) of $\hat{v}$ against the relaxation value v_r, on three test systems spanning two orders of magnitude in size.

Table 5 varies network depth at fixed width. Prediction error falls steeply as the network deepens—for `case118`, MAPE drops from 8.7% with a single hidden layer to 3.1% with six—and largely plateaus beyond four to six layers, while training cost grows only modestly. We adopt six hidden layers as a point past which additional depth yields diminishing returns. Table 6 varies the number of training epochs for the chosen 6×256 architecture. Error decreases monotonically with the training budget and flattens by roughly 1000 epochs (e.g. `case118` improves from 1.38% at 1000 epochs to only 1.29% at 2000), so we train for 1000 epochs throughout. Both trends were consistent across the three test systems, and we found the same architecture transferred well to the other problem families without further tuning.

Table 5: Network-*depth* ablation on AC-OPF (SOCP relaxation): 4-fold cross-validated MAPE (%) of $\hat{v}$ against v_r, at fixed width 256 and 1000 epochs. Error falls steeply with depth and plateaus by 4–6 hidden layers, which motivates our choice of six. Train time is for `case1354` (seconds, all four folds).

Hidden layers	`case9`	`case118`	`case1354`	**Train (s)**
1	2.92	8.68	5.87	127
2	1.19	6.93	1.83	181
3	1.07	4.65	1.05	208
4	1.04	3.74	0.74	244
5	–	3.43	0.74	237
6	–	3.12	0.72	267

Table 6: Training-*budget* ablation on AC-OPF (SOCP relaxation, 6×256 network): 4-fold cross-validated MAPE (%) of $\hat{v}$ against v_r versus the number of training epochs. Error decreases monotonically and largely plateaus by 1000 epochs, our chosen budget. Train time is for `case1354` (seconds, all four folds).

Epochs	`case118`	`case1354`	**Train (s)**
70	4.61	1.11	90
200	3.26	0.77	256
400	2.33	0.53	517
700	1.58	0.50	630
1000	1.38	0.45	799
1500	1.35	0.45	1256
2000	1.29	0.46	1638